

HOSTED BY



FIGHT NIGHT

BLUE CORNER

DEVELOPER

STOP **FIGHTING** YOUR
SPRING BOOT TESTS

TUE 22 SEP 2026 · 2 PM UTC+2

Software Craft Community @ DATEV

RED CORNER

SPRING BOOT
TEST SUITE

VS

Referee: Philip Riecks

Achieve Confidence in Every Commit





Stop **Fighting** Your Spring Boot Tests

Why your tests fight back - and how to make
them fight for you

Software Craft Community @ DATEV · September 22, 2026

About Philip

- Self-employed developer from [Herzogenaurach](#), Germany (close to Nuremberg) 🍺
- Met Sven Hansen during the [Spring I/O](#) conference in Barcelona this year 🇪🇸
- Blogging & content creation with a focus on testing Java and specifically Spring Boot applications 🌱
- Founder of [PragmaTech GmbH](#) - We Help Developers to Frequently Deliver Software with More Confidence



Participate During the Talk

Go to menti.com and use the code **7735 0833** to **anonymously** submit answers:



Please answer the **first three questions**:

- Who wrote your tests last week?
- How long does your test suite take end to end?
- How confident are you deploying on a Friday afternoon (0 to 10)?

Why Test Software?



**In 2026, most of our code is not
written by us.**



Frog told the LLM to write tests for all the code it wrote.

“There,” he said. “Now the model can’t write broken or buggy code.”

“But the model can write broken or buggy tests,” said Toad.

“That is true,” said Frog.

My Overall Northstar for Engineering Excellence

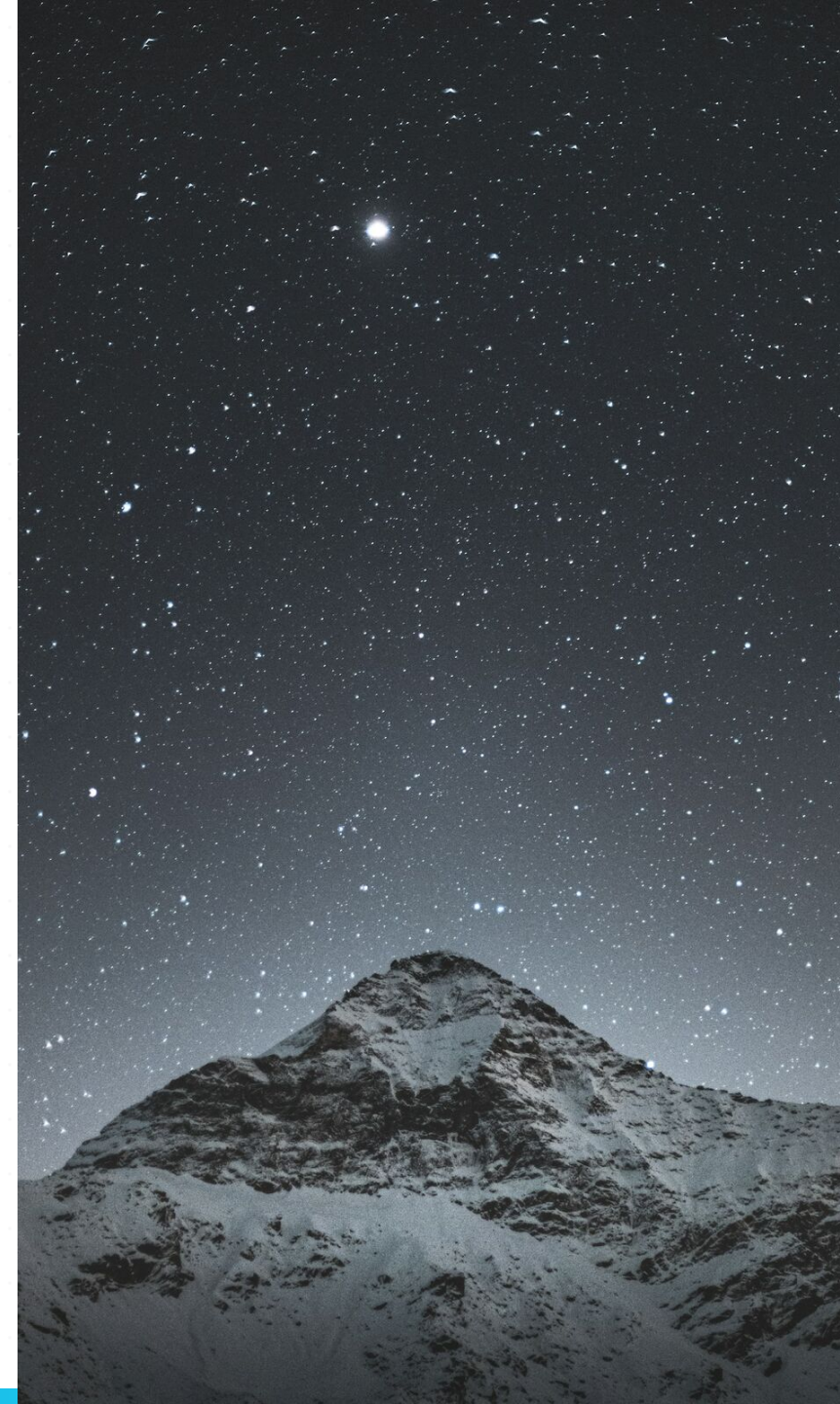
Imagine seeing this pull request on a Friday afternoon:

☐  Update dependency org.springframework.boot:spring-boot-starter-parent to v4.0.0 ✓

 3

#419 opened on Oct 31 by renovate  · Review required  1 task

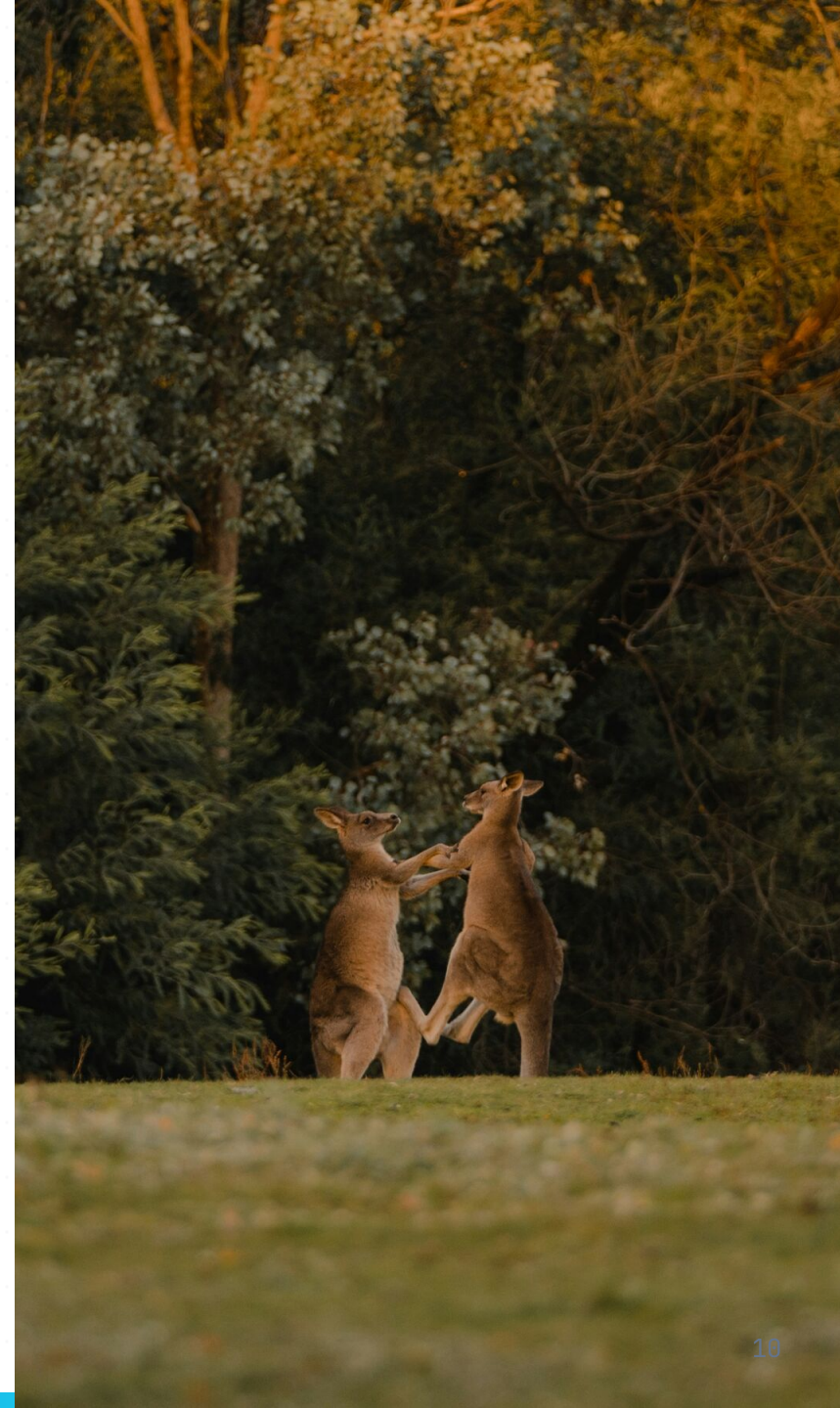
How confident are you to merge this major Spring Boot upgrade and deploy it to production once the pipeline turns green?



**Good tests don't just catch bugs -
they give you [fast feedback](#) and
[confident deployments](#).**

When Your Spring Boot Tests Fight Back

- The suite takes > 30 minutes, so nobody runs it before pushing
- A test goes red and you cannot tell whether the code or the test is broken
- You reach for `@SpringBootTest` because it is the only thing that works
- The pipeline is green and you still do not want to deploy on a Friday



**It is almost never the framework.
It is **three myths** we keep
repeating.**

The Three Myths We Bust Today

01 "I need @SpringBootTest for that."

02 "Spring Boot tests are slow."

03 "It is green, so it works."

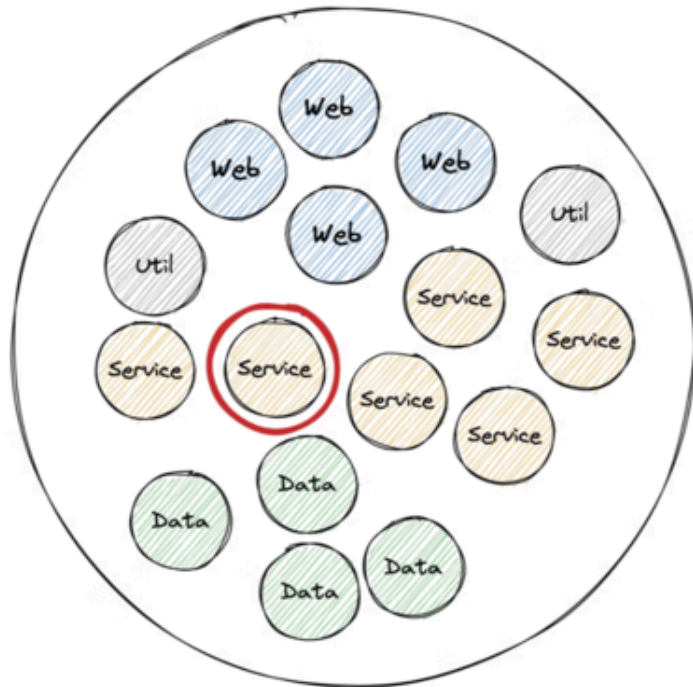
MYTH #1

“Every test needs the full context.”

Spoiler: you don't always need `@SpringBootTest`.

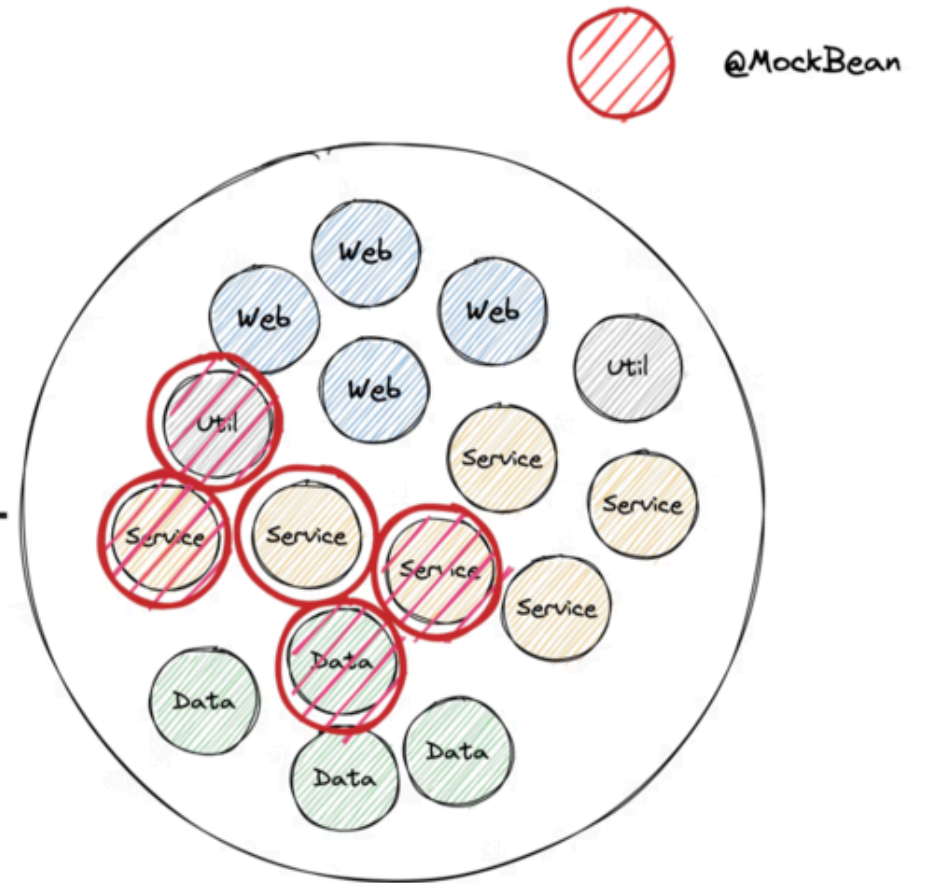


The `@SpringBootTest` Obsession



Testing every
single class with
`@SpringBootTest`

... results in a lot
of different
context setups
and slows down
the build



Two Families of Tests

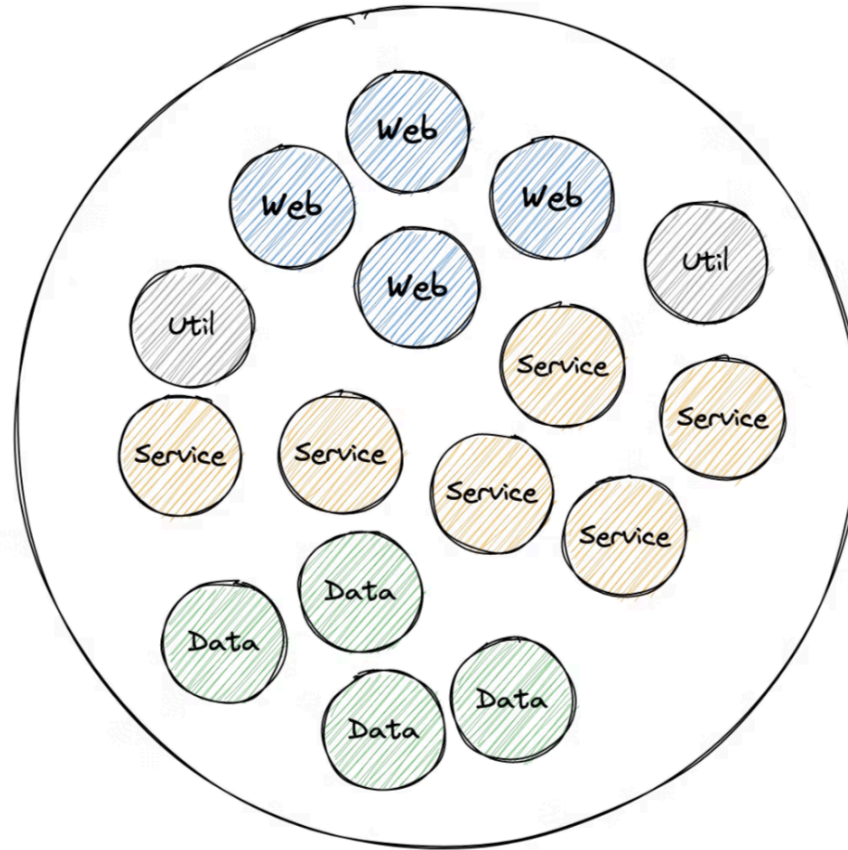
Without a Spring context - plain JUnit and Mockito. Milliseconds per test. Your business logic, your algorithms, your edge cases.

With a Spring context - Spring wires the beans for you. Seconds per context start. Needed when the framework itself is part of the behaviour you verify.

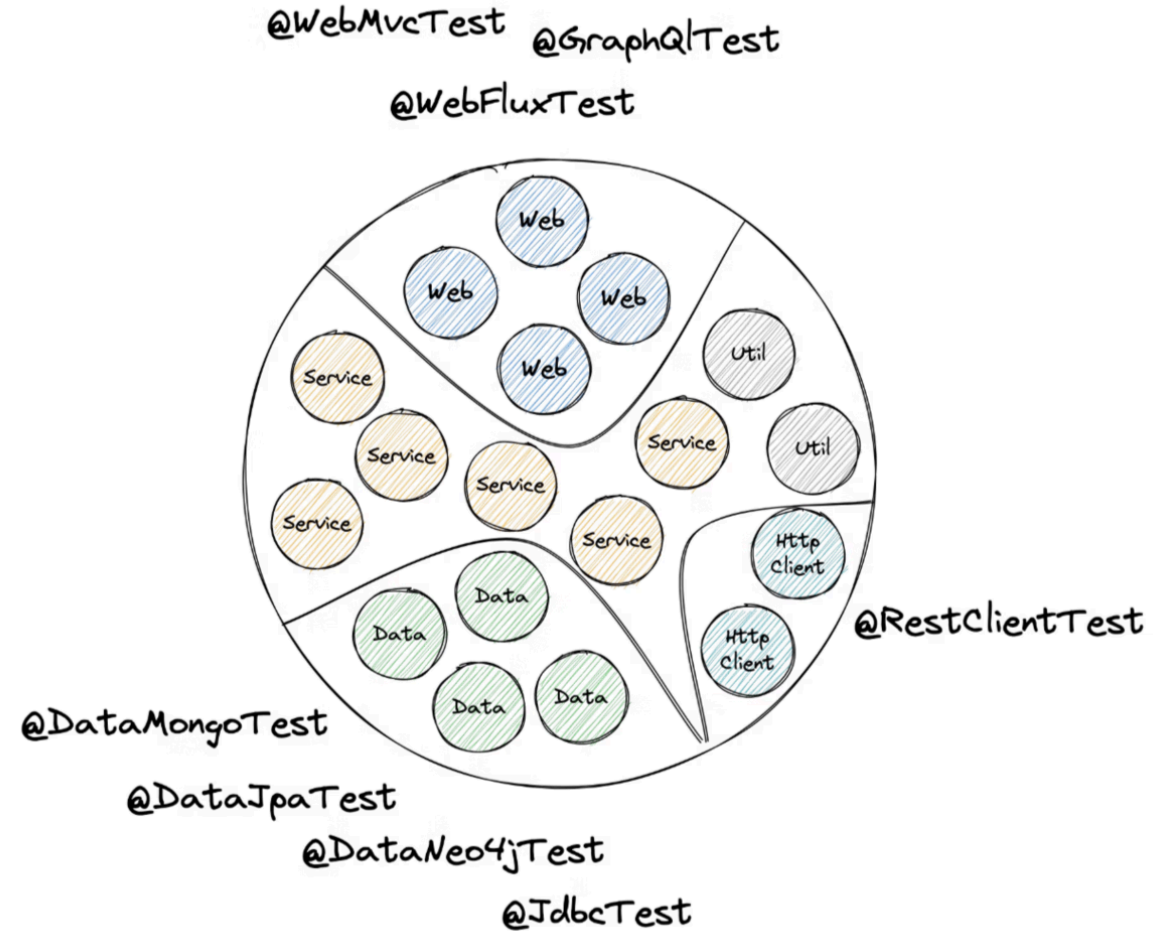
The second family splits again:

- **Sliced context** - only the layer under test (`@WebMvcTest` , `@DataJpaTest` , ...)
- **Full context** - the whole application (`@SpringBootTest`)

A Typical Spring `ApplicationContext`



You Don't Always Need the Entire Context



Family 1: No Context At All - Milliseconds

```
1  @ExtendWith(MockitoExtension.class)
2  class CustomerServiceTest {
3
4      @Mock
5      private CustomerRepository customerRepository;
6
7      @InjectMocks
8      private CustomerService customerService;
9
10     @Test
11     void shouldCreateNewCustomerWhenNameDoesNotExist() {
12         when(customerRepository.findByCustomerName("duke")).thenReturn(empty());
13
14         String customerId = customerService.createNewCustomer("duke");
15
16         assertThat(customerId).isEqualTo("42");
17     }
18 }
```

Things We Can't Cover Without a Context

- **Request Mapping:** Does HTTP GET `/api/customers/{id}` actually resolve to our desired method?
- **Validation:** Will an incomplete request body result in a 400 Bad Request or an accidental 201?
- **Serialization:** Are our JSON objects serialized and deserialized correctly?
- **Headers:** Are we setting `Content-Type` or custom headers correctly?
- **Security:** Are our Spring Security configuration and authorization checks enforced?

This is where a context earns its cost - and a sliced one is usually enough.

Family 2: A Sliced Context

```
1  @WebMvcTest(CustomerController.class)
2  @Import(SecurityConfig.class)
3  class CustomerControllerTest {
4
5      @Autowired
6      private MockMvc mockMvc;
7
8      @MockitoBean
9      private CustomerService customerService;
10
11     @Test
12     @WithMockUser
13     void shouldReturnLocationOfNewlyCreatedCustomer() throws Exception {
14         // web layer only: mapping, validation, serialization, security
15     }
16 }
```

Decision Paralysis: When to Include a Context

A simplified decision table:

Question	Tool
Does my business logic work?	Plain JUnit + Mockito, no Spring
Does my HTTP layer map, validate, serialize?	<code>@WebMvcTest</code>
Does my query return what I think?	<code>@DataJpaTest</code>
Does my client talk to the remote API?	<code>@RestClientTest</code> (including a mock HTTP server)
Does the whole thing start and work end to end?	<code>@SpringBootTest</code>

**Start at the level that answers
your question. Climb only when
the test forces you to.**

MYTH #2

“Spring Boot tests are slow.”

You are not paying for tests. You are paying for context starts.



Slow Tests Are Not the Problem. Slow [Feedback](#) Is.

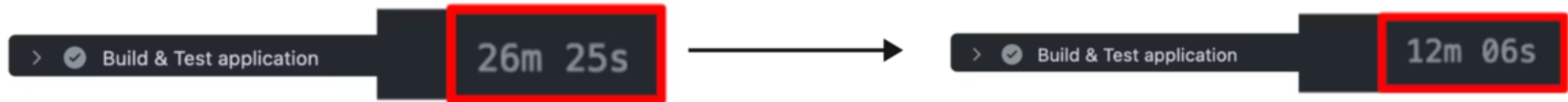
Attack it from four angles:

- 1. [Right test level](#)** - the cheapest test that answers the question (Myth #1)
- 2. [Fewer context starts](#)** - Spring already caches contexts, if you let it
- 3. [Parallel execution](#)** - use the cores you are paying for
- 4. [Reused infrastructure](#)** - stop restarting Docker containers (Myth #3)

Angle 2: Spring Context Caching

- **The problem:** integration tests need a started and initialized `ApplicationContext`, which takes time
- **The solution:** the Spring TestContext framework caches a started context for later reuse
- Part of every Spring Boot project already, via `spring-boot-starter-test`

Speed improvement example:



How the Cache Key is Built

```
1 // DefaultContextCache.java
2 private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LruCache(32, 0.75f));
```

This goes into the cache key (`MergedContextConfiguration`):

- activeProfiles (`@ActiveProfiles`)
- propertySourceProperties (`@TestPropertySource`)
- contextCustomizer (`@MockitoBean` , `@MockBean` , `@DynamicPropertySource` , ...)
- etc.

Every unique combination is a new context start.

The Final Boss

Developers consult AI and StackOverflow for integration test issues and copy advice without knowing the implications:

```
1  @SpringBootTest
2  @DirtiesContext
3  // this instructs Spring to remove the context from the cache
4  // and rebuild a new context on every request
5  public abstract class AbstractIntegrationTest {
6
7  }
```

The setup above **disables** context caching and slows the build down significantly.

New in Spring Framework 7: Pausing of Test Contexts

“

As of Spring Framework 7.0, we now pause test application contexts when they are not used.

An application context stored in the context cache will be stopped when it is no longer actively in use and automatically restarted the next time the context is retrieved from the cache.

Angle 3: Test Parallelization

Goal: use the cores you already pay for.

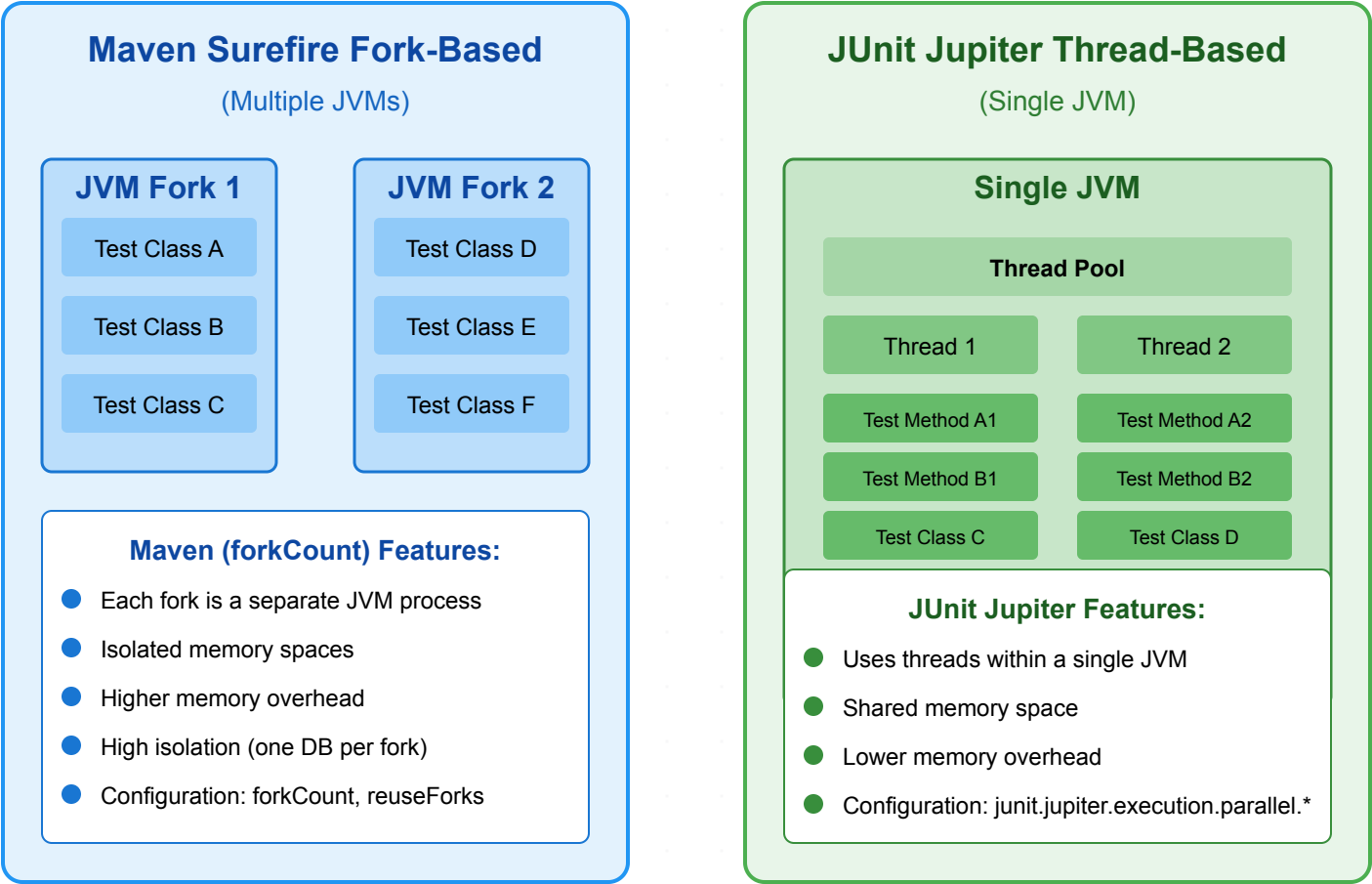
Requirements:

- No shared state
- No dependency between tests or their execution order
- No mutation of global state

Two ways to get there:

- Fork a new JVM with Maven/Gradle - more resources, fully isolated
- Use JUnit Jupiter's parallel execution mode - same JVM, multiple threads

Java Test Parallelization Options



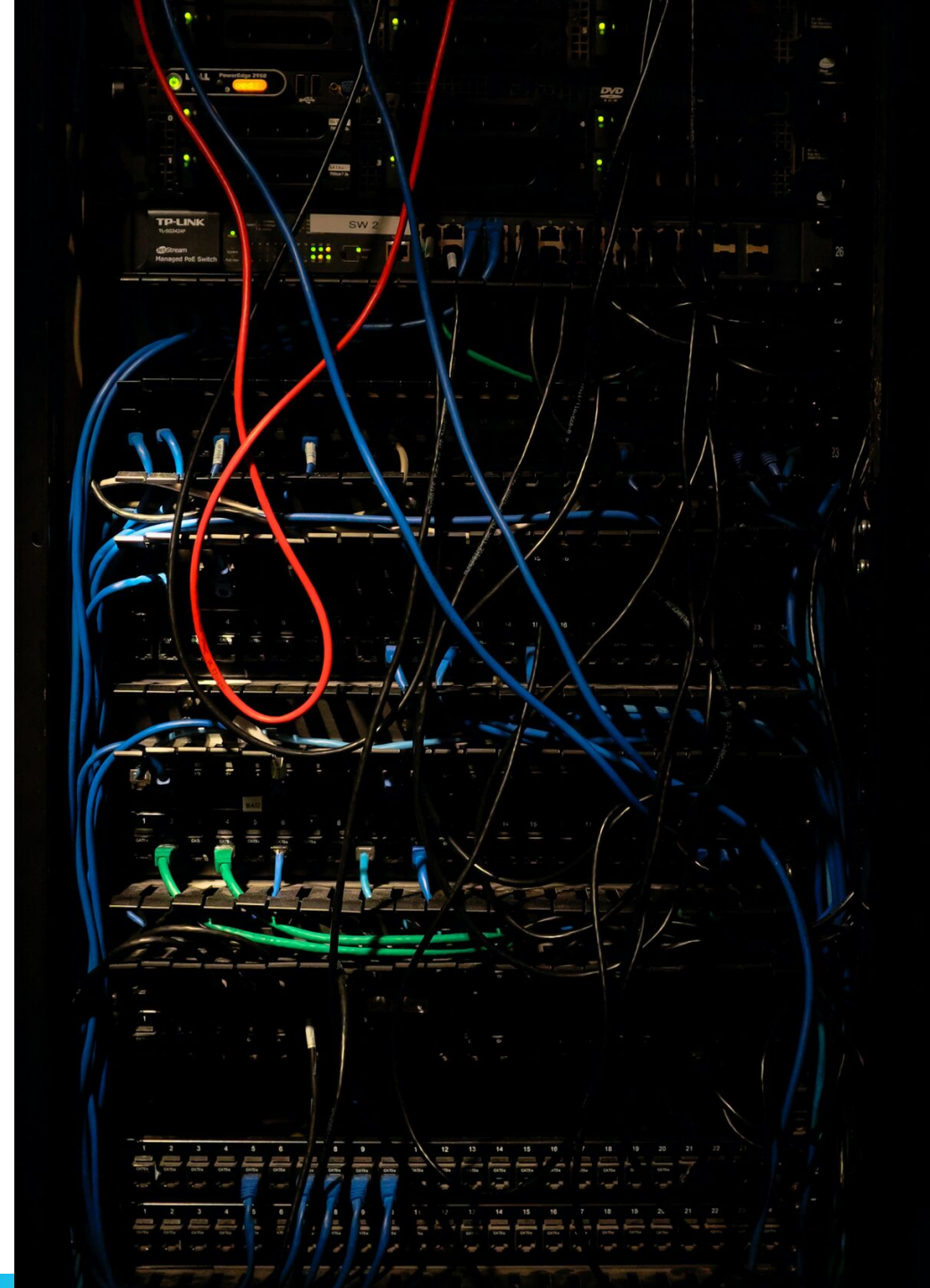
Make the Most of Both

- Avoid `@DirtiesContext`, especially in central base classes
- Understand how the cache key is built before you add another annotation
- Monitor and investigate context restarts instead of guessing
- Align the number of unique context configurations across the suite
- Then, and only then, turn on parallel execution

MYTH #3

“It is green, so it works.”

Green proves your tests ran. Not that they would have caught anything.



Where Tests Drift Away From Production

- **The database:** H2 in tests, PostgreSQL in production. Different SQL dialect, different constraints, different behaviour.
- **Remote services:** mocked away with Mockito, so the HTTP layer, the serialization and the error handling are never exercised.
- **The test itself:** it asserts that the code ran, not that the code is right.

Each gap is a place where a green build still ships a bug.

Must-Have #1: Testcontainers (Real Infrastructure)

Use case: your test needs the real database, broker or cache - not an in-memory stand-in.

```
1 @Container // <-- Testcontainers manages the lifecycle of the container
2 @ServiceConnection // <-- automatically configures Spring Boot datasource properties
3 static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine")
4     .withDatabaseName("testdb")
5     .withInitScript("init-postgres.sql");
```

Same engine, same version, same dialect as production. Thrown away after the run.

Keep Testcontainers Fast

- Enable container reuse where possible: `.withReuse(true)`
- Prefer a singleton container per test run over `@Testcontainers` (which starts one per test class)
- Speed up startup with prebuilt images that already contain your schema and seed data

```
1  private static PostgreSQLContainer<?> postgres =  
2      new PostgreSQLContainer<>("myteampostgres:42");  
3  
4  static {  
5      postgres.start(); // started once, shared by every test that needs it  
6  }
```

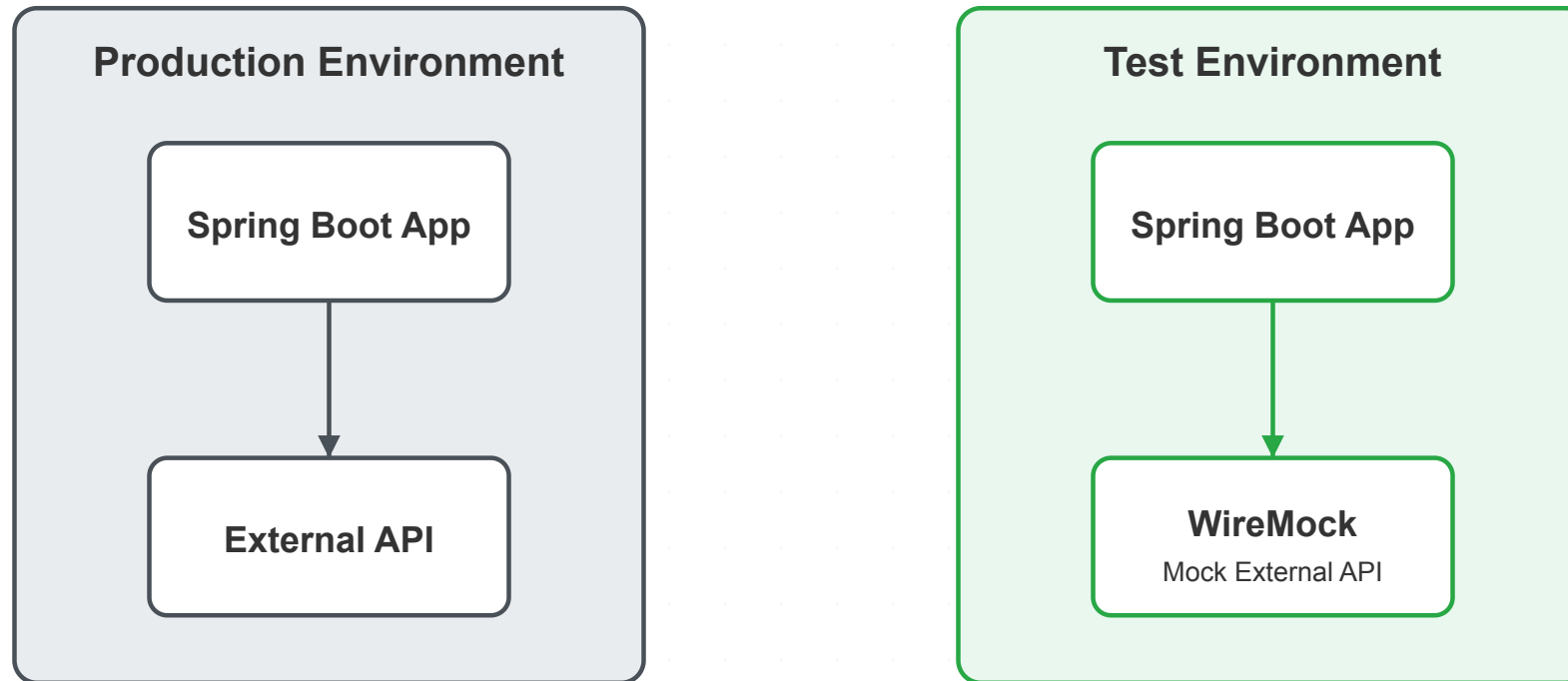
Must-Have #2: WireMock (Real HTTP)

Use case: your application calls a remote API. You want to exercise your own HTTP client, serialization and error handling - without depending on someone else's uptime.

Mocking the client with Mockito skips exactly the part that breaks in production:

- Wrong URL or query parameter
- Unexpected response shape
- Timeouts, 500s, and retry behaviour

WireMock: HTTP Service Stubbing for Testing

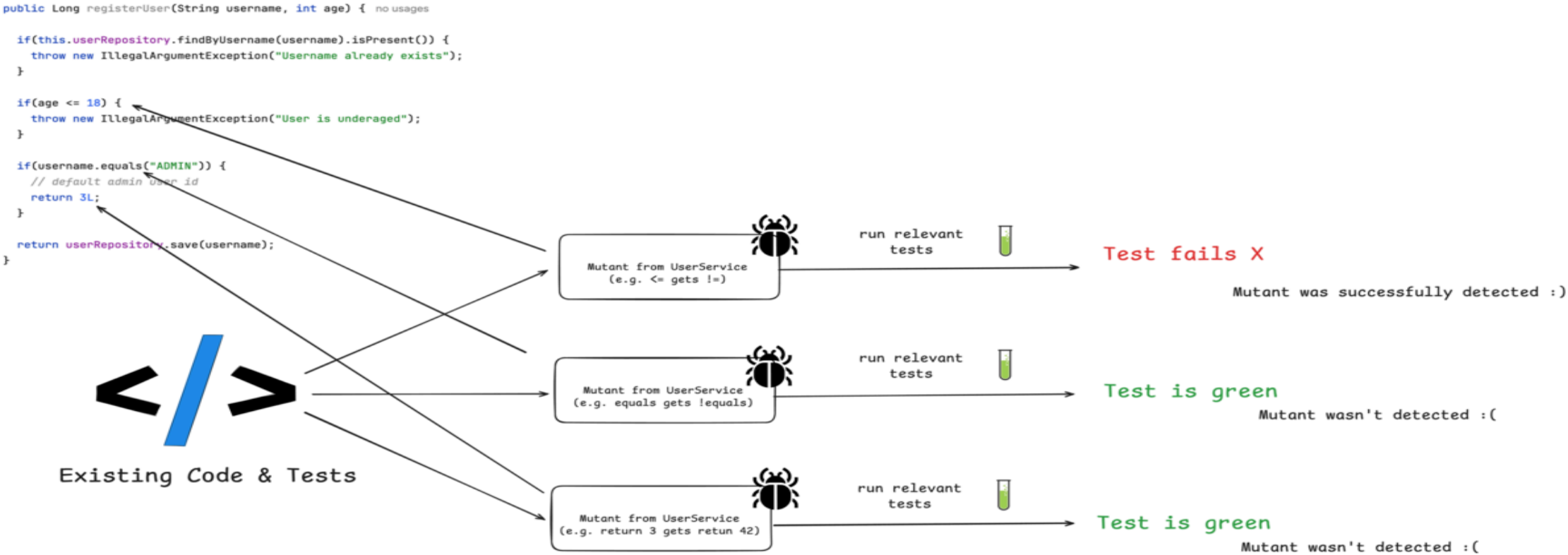


Key Benefits of WireMock

- Controlled test environment
- No network dependency
- Predictable responses
- Simulate error conditions
- Fast test execution
- Java integration

Must-Have #3: Mutation Testing (Real Assertions)

- High code coverage gives you a **false sense of security**: it shows which lines ran, not which bugs you would catch
- **PIT mutates your source code** - flips conditionals, changes return values - and re-runs your tests
- If a mutation survives, no test noticed the change. That is a **blind spot** with high line coverage on top of it
- A mutation score threshold in CI is a quality gate that coverage can never be



The Three Myths, Busted

- 01 Most tests need no Spring context - climb only when forced to
- 02 Your suite is not slow, it is restarting contexts - cache, align, parallelize
- 03 Green is not proof - real infrastructure, real HTTP, real assertions

Coming Up: Ship Fast, Sleep Well

Fearless Spring Boot Deployments in the AI Era - my next talk at the **DATEV Coding Festival**. We pick up where today stops: shipping the code an agent wrote, without losing sleep over it.

Come and join:

Oct 6

2026

14:30

to 15:15

DATEV

Coding Festival

What I Do to Build Confidence in the Agentic Coding Era

- **Review generated tests** like a pull request from a new hire
- **Make the rules executable**: ArchUnit rules, testing conventions in `CLAUDE.md` / `AGENTS.md`, mutation score thresholds in CI
- **Custom skills**: define once per project how testing is tackled, with best practices and antipatterns
- **Keep the feedback loop tight**: fast, trustworthy tests let the agent iterate without me watching every step

My Agentic Testing Setup for Spring Boot

Define a **skillset** for fast and comprehensive tests, including a **test strategy** for the given project:

```
1  .claude/skills/  
2  |— unit-testing      fast tests without context bloat  
3  |— slice-testing     right-sized Spring context slices  
4  |— slice-testing-webmvc web layer with @WebMvcTest  
5  |— integration-testing full-context tests that stay fast  
6  |— e2e-testing       user journeys against the running app  
7  |— testcontainers-setup real infrastructure, reused containers  
8  |— test-setup-reviewer flags test anti-patterns for you
```

Each skill carries rules, references, best practices and antipatterns, described as code and text.

You can delegate the typing.

You can't delegate the ownership.

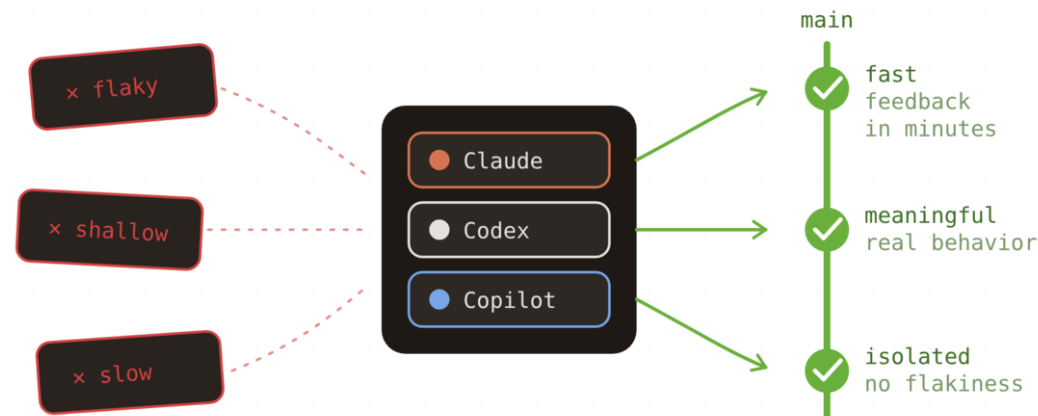
You get paged at 3 AM.

**Invest in a test suite that gives
you confidence in every commit.**



Get Notified

I am building an **agentic Spring Boot testing course**: how to make code agents produce tests you would have written yourself.



Agentic Testing for Spring Boot

From AI-generated noise to confidence in every commit



Joyful Testing!

Get notified for Agentic Testing for Spring Boot:



- [LinkedIn](#) (Philip Riecks)
- [Mail](mailto:philip@pragmatech.digital) (philip@pragmatech.digital)

